

A data queue with quarantine: proposal for online data selection in GoLLeM-v6

Arkadiusz Słota*

26 September 2026 — proposal, no results yet

Abstract

The training pool is made larger than the training budget and fed to the trainer as an ordered queue of shards. After each shard the model is measured on low-noise control losses, one per text domain. A shard after which a control loss rises above a threshold fixed before training is rolled back and gets a second chance in the next pass over the pool; after a second rejection it moves to a quarantine; the next shard is taken from the queue. Quarantined shards are analysed (source, register, duplicates, test overlap, language) and either returned to the queue or dropped. The proposal follows from three observations in GoLLeM-v5: late changes of the mixture moved the board score by no more than the noise, single checkpoints move by ± 0.4 eff with the recipe unchanged, and in the one from-scratch comparison we have, the WikiText loss separated two corpora by about fourteen times its seed-to-seed spread while the board score did not. We describe the mechanism, the measurement, a falsifiable experiment and the risks. Nothing here is a result.

1 Motivation: what GoLLeM-v5 taught us about data decisions

Mixture tweaks are below the noise of the board score. In round 3 of v5 two arms continued the same run with different mixtures (our ARC-oriented mixture with FineWeb-Edu added 45:55, versus the same mixture with its question-answer documents doubled). At 400k steps the paired difference was $\Delta_{\text{eff}} = +0.11$ $[-0.30, +0.51]$, and the sign changed between neighbouring checkpoints (+0.46, -0.13, -0.02, +0.11). The composition decision for the final runs (Z1) was a tie as well: $\Delta = -0.24$ eff against a threshold of 0.6. Continuing the finished 64M model in segments on new data (S12) gave -0.24 $[-0.68, +0.20]$.

A single checkpoint moves by about half a point with the recipe unchanged. Between steps 120k and 140k of the final 64M run the board score fell by 0.42 eff. A paired bootstrap over the same questions gives $[-0.96, +0.12]$; the model changed its ARC-Easy answer on 337 of 2376 questions (158 for the better, 179 for the worse). A rule of the form “drop the data if the score fell” would therefore discard shards at random.

Domain losses do separate data choices. In the same Z1 comparison the two seeds of arm Z differed in WikiText byte perplexity by 0.011, while arm Z differed from arm C by 0.155–0.166, about fourteen times the seed spread; on eff the arms differed by less than the threshold. This is one comparison of two very different corpora; whether domain losses also separate smaller data differences reliably is exactly what Stage A of the experiment (Section 4) measures.

The protocol matters as much as the data. Prepending a document separator, so that the first token of each BLiMP sentence is also scored, moves our 64M model from 75.83 to 78.56 without any training. Every number used for a data decision must come from one fixed protocol on a held-out selection split, never from the board test set.

*Prepared with the Kolektyw AI agents: Hart (draft), Latarnik (editing), Monter (implementation and costs), Wartownik (review).

2 Mechanism

1. **Pool and queue.** The candidate pool (after the licence, PII and decontamination gate) is larger than the token budget, e.g. $1.5\times$. It is cut into shards of one checkpoint interval (0.66 B tokens on the v5 trainer; Section 5), each labelled with its source, language and register mix. Shards are ordered by a seeded shuffle within the planned mixture proportions.
2. **Control sets.** Four to six fixed, deduplicated and decontaminated sets of $\approx 1\text{--}2$ M tokens: EN educational, EN encyclopaedic, EN question–answer, PL general, PL encyclopaedic, code/maths if in scope. None of them overlaps the pool or any benchmark (checked with the existing n-gram and semantic scanners).
3. **Measurement after each shard.** Per-domain loss (bits per byte) on every control set; a few minutes in total (Section 5).
4. **Quarantine rule (fixed before training).** Let σ_d be the shard-to-shard spread of the change in loss on domain d , measured on the first k shards drawn from one homogeneous source. A shard is quarantined if, after it, the loss on any domain rises by more than $3\sigma_d$ and does not recover within the next shard. The model state is rolled back to the checkpoint before the shard; the shard goes to quarantine and the shard used to test recovery goes back to the front of the queue, so a rejection costs two shards of compute. Because a shard is judged on several domains at once, the σ -level and the recovery condition are set so that the false-rejection rate measured in Stage A is below 5%.
5. **Second chance in the next rotation, then quarantine (idea: A. Słota).** The queue is consumed in rotations (passes over the pool). A shard rejected once moves to the end of the *next rotation*: it returns in the stable phase, at a later stage of the model, and faces the rule again. A shard rejected twice, at two different stages, goes to quarantine: two rejections are a signal rather than noise. Rejected shards never enter the final learning-rate decay, which uses accepted shards only; the low learning-rate end is where v5 gained most, so it is the worst place for doubtful data. With a pool much larger than the budget (e.g. 100B candidates for 30B of training), the end of the candidate queue is in practice a quarantine: those shards are used only if better ones run out.
6. **Analysis.** For each quarantined shard: source and language shares, register distribution (the HPLT curation pipeline), duplicate rate, test overlap, length and boilerplate statistics. Shards with a found cause are dropped or the cause is filtered at source; shards without one return to the queue once.
7. **Board-style checks.** ARC-Easy, BLiMP and WikiText are measured every few shards only to confirm that control losses track quality; they never trigger quarantine.

Curve-guided variant of the rule (idea: A. Słota). A rise in control loss catches harmful shards but not shards that simply do not help. In the stable phase each domain loss follows a smooth curve in the number of tokens seen, so a reference curve $\hat{L}_d(D) = a_d + b_d D^{-c_d}$ can be fitted, in the stable phase only, on the shards of Stage A (a known-good source) and the rule applied to the *residual*: a shard is quarantined if its measured decrease in loss falls short of the decrease predicted by $\hat{L}_d$ by more than $k\sigma_d$, on two consecutive shards or on several domains at once. Unlike efficiency, domain losses are smooth enough for such fits (in v5 the WikiText curve of the final 64M run is identified, asymptote 2.352 [2.335, 2.367], while the efficiency curves are not). Three safeguards. (i) The curve rule is used only once the fit passes the identifiability check of the v5 figures (narrow interval for the asymptote, exponent not at a bound); until then the simpler rise rule applies. (ii) The reference curve is frozen after Stage A; if it is refitted on a schedule written before training, the refit uses the random-order arm A, never the accepted shards of arm B, which would teach the curve to expect what the queue already let through. (iii) A stop condition written in advance: if the rejection rate exceeds a fixed share or the remaining pool falls below the remaining budget, the rule switches off and the queue runs in plain order. An aspirational curve, the trajectory we would like to reach, can be drawn as a target but must not serve as the threshold: set too high, it would reject almost everything and keep the shards most similar to the control sets.

Reference curves from earlier models (idea: A. Słota). GoLLeM-v5 and earlier runs already give domain-loss trajectories. Re-measured on the same control sets, they form a “historian”: the shape of the curve, the typical rate of improvement and how fast data are used up, for each size and recipe. They inform the form of $\hat{L}_d$ but not its parameters, which come from Stage A of the run itself, because a different size, schedule or pool

gives a different curve. Only runs with the same schedule type, tokenizer and protocol are comparable, as with Figure 1 of the v5 report.

Two tiers: screen cheaply, train selectively. Judging a shard by training on it costs the training compute of that shard even if it is rejected, so a pool of 100B candidate tokens judged this way costs 100B tokens of training even if only 30B are kept. The goal “pass over 100B, learn from 30B” therefore needs a cheap first tier: forward-only scoring of every shard by a small reference model (loss, and its residual against $\hat{L}_d$), which costs a fraction of training, followed by the training-based quarantine rule only for shards that pass the screen. Whether the screen predicts the second tier is itself measured in Stage A.

A by-product: data-quality labels (observation: A. Słota). Every shard ends with a record: its source and register mix, its screen score, its measured residual against $\hat{L}_d$ and the decision. Over a run this is a labelled set of data quality *relative to this model and training stage*, which can be aggregated per source for the dataset catalogue and could later train a cheap quality classifier. The labels are model- and stage-dependent, so they are evidence for the catalogue, not a universal ranking.

Adding data (idea: A. Słota). New data join at the end of the next rotation, never in the middle of one, so the composition, the reference curve and the threshold stay valid within a rotation. Each new source passes the licence, PII and decontamination gate before it enters the queue, and its first shards run as a small Stage A (its own σ_d), because the first shards of a new source always raise other domain losses for a while. Every addition creates a new version of the queue manifest (hash, date and the list of added shards with their hashes), written before any of the new shards is trained on.

Learning-rate schedule. The queue runs only in the stable phase of a warmup–stable–decay schedule: there every checkpoint is a valid rollback point and the learning rate, hence the size of a shard’s effect, is constant. The final decay uses accepted shards only, never quarantined ones. Under a cosine schedule the threshold would drift with the learning rate.

New domains. The first shard of a new source (e.g. the first Polish shard) always raises the loss on other domains for a while. Thresholds are therefore calibrated per source, and a shard is judged with a delay of one shard (the recovery condition in step 4).

3 Why this measurement

signal	noise, same recipe	effect of a data change
board eff (one checkpoint)	± 0.4 (64M, 120k→140k, paired CI width 1.08)	≤ 0.5 (mixture tweaks)
WikiText byte ppl	0.011 (two seeds, Z1)	0.155–0.166 (Z vs C)

Table 1: Why quarantine decisions use domain losses. Sources as in Section 1.

4 Experiment (falsifiable)

On a small proxy (the 64M shape of v5, same trainer) with the same token budget:

- **Stage A, noise and reference:** 10 shards from one homogeneous source; estimate σ_d per domain and fit the reference curves $\hat{L}_d$ used by the curve-guided rule.
- **Stage B, three arms, same token budget, two seeds each:** A = plain seeded order of the pool; B = queue with quarantine (rise rule or curve-guided rule, chosen before the run); C = queue that rejects *random* shards at the same rate as B. Arm C controls for the mere act of rejecting (less diverse data, different order).

- **Decision rule (written before the run):** the queue is adopted for v6 only if, after the same final decay, B beats both A and C on the selection axis (EN and, once defined, PL) by more than $\max(0.6, 2 \times \text{seed spread})$ eff or its per-axis equivalent. Control losses are *not* a success criterion, because B optimises them by construction. Otherwise the queue is kept only as an analysis tool (the quarantine list), not as a training rule.
- **What we learn either way:** the list of shards that raise domain losses and why, which is the input for choosing EN and PL sources for v6.

5 Implementation on our pipeline

What exists today. Training pools are flat `uint16` token streams with an end-of-text separator between documents. Each pool comes with a document index (start offsets) and a manifest holding the SHA-256 of the pool and of every input. The trainer (r6) draws training windows with an epoch sampler. Every window is used once per pass, and a resume from a checkpoint reproduces the continuous run exactly, because the sampler state is a function of the seed and the step. Checkpoints are written every 20 000 steps ($20\,000 \times 32 \times 1\,024 \approx 0.66\text{B}$ tokens), uploaded, and deleted locally only after their hash matches the copy on the Hub. Pools are compacted in place with asserted document and token counts.

Queue. The pool is split into *shards* of equal token count, each a document-aligned slice with its own hash and source label. A shard of one checkpoint interval (0.66B tokens) keeps the bookkeeping aligned with checkpoints. The queue is an ordered list of shard ids in a small manifest file. It is hashed and versioned like every other input. In the trainer, the epoch sampler iterates over the shards in queue order instead of over one flat file. The queue position becomes part of the checkpoint (next shard id + offset inside it), so resume stays exact. The total queue is larger than the training budget, so rejected shards can be replaced. A pass over the queue is a rotation with a frozen manifest: its list of shard hashes does not change while the rotation runs, which keeps resume exact. Shards given a second chance and newly added shards (after the data gate) enter the manifest of the next rotation; every manifest version is hashed and recorded before training on it starts.

Quarantine step. At the end of each shard the trainer writes a checkpoint, and the control losses are computed (Section 3). If the rule marks the shard, then:

1. the shard id moves from the queue to a quarantine list, recorded with the reason and the measured values;
2. training resumes from the checkpoint written *before* that shard;
3. the next shard in the queue takes its place.

The rejected segment costs one shard of compute, or two when the recovery condition of the rule is used. Because resume is exact, the only difference between the kept and the discarded trajectory is the data of that shard. That is the property the method relies on.

Branch check. For the first rejections, the same checkpoint is also continued with a different shard from the same source, as a short branch on a second GPU. If the replacement shard shows the same increase, the problem is the phase of training and not the shard, and the shard returns from quarantine. This is the branching procedure used in v5 (runbook RB4), applied per shard.

Cost and resources (from v5 measurements).

- A 64M model on one RTX 5090 runs at about 204k tokens/s, so one 0.66B-token shard takes about 55 minutes and a 128M shard about 78 minutes.
- Control losses on a few million tokens take minutes on the same GPU; they replace board evaluations, which take about 15 minutes per checkpoint and are too noisy for this decision.
- Each rejection adds one or two shards of compute; the branch check adds one shard on a second pod.
- A queue of 25B+ tokens in `uint16` needs about 50 GB, so it needs a volume rather than the 30 GB container disk used in v5. Shards can also be streamed from the Hub at about 15 MB/s per pod, i.e. about 1.5 minutes per 1.3 GB shard, well below the time to train on it.

What has to be built.

- a shard builder (split and hash; reuses the in-place compaction tooling);
- queue support in the sampler, with the queue position stored in the checkpoint;
- the rule and the quarantine list as a separate supervisor step, next to the out-of-memory supervisor used in v5.

Each part goes through the same gate as the v5 pipeline: script hash reviewed before it runs, and a test that injects a rejected shard and checks that training resumes from the earlier checkpoint with the next shard.

6 Risks

- **Short-term versus long-term effects.** A new language or domain can raise other domain losses for a while and still help later. Hence the recovery condition and the one return from quarantine.
- **Goodhart.** Many accept/reject decisions optimise the model towards the control sets. They are therefore disjoint from every benchmark and from the selection split, and the final comparison uses the selection split only.
- **Drift of the threshold.** σ_d shrinks as training proceeds; it is re-estimated on a fixed schedule written before training, not after a surprising shard.
- **Reproducibility.** The queue manifest (order and hash of every shard), the quarantine list with reasons and measured values, and every return from quarantine are logged; without this nobody can rebuild the model or check that the rule was not bent after a result.
- **Gate before the queue.** Licence, PII and decontamination are checked before a shard enters the queue; quarantine is for data quality, not for legal problems.
- **Cost.** Overhead $\approx$ twice the rejection rate (a rejected shard's compute is lost) plus a checkpoint per shard and the control-loss evaluation (minutes); see Section 5.
- **Related work (ideas only).** RHO-LOSS selects examples by their loss on a held-out set; DoReMi sets domain weights with a proxy model; Online Data Mixing treats domains as bandit arms; DataDecide asks whether small-scale data decisions predict large-scale ones. Our difference is modest: decisions at the level of whole shards with rollback, pre-registered thresholds, and a quarantine analysis loop that feeds source selection.